and to handle the complexity created by having many components instead of one.

NOTE This book is not about microservices, so I won't go into detail about them. If you are interested in the topic, you can check out *Building Microservices*, second ed., by Sam Newman (O'Reilly, 2021) and *Microservices Patterns* by Chris Richardson (Manning, 2018). For a more Spring-oriented analysis, you can find *Spring Microservices in Action*, second ed., by John Carnell and Illary Huaylupo Sanchez (Manning, 2021) in the Manning catalog. If you are not familiar with microservices, don't worry. That knowledge is not required to follow along with this book.

After many years of fame and failed migrations, intense discussions about the future of this popular architectural style have arisen in the developer community. Some engineers started talking about *macroservices* to reduce the number of components and, therefore, the complexity of managing them. The term "macroservices" was suggested sardonically by Cindy Sridharan, but it has been adopted in the industry and used by companies like Dropbox and Airbnb to describe their new architectures.[16] Others proposed a *citadel* architectural style, consisting of a central monolith surrounded by microservices. Still others are advocating a return to monolithic applications in the form of modular monoliths.

In the end, the important thing is to choose an architecture that can deliver value to our customers and our businesses. That's why we develop applications in the first place. Each architectural style has its use cases. There is no such thing as a silver bullet or one-size-fits-all solution. Most of the negative experiences related to microservices were caused by other issues, such as bad code modularization or unfitting organization structure. There shouldn't be a battle between monoliths and microservices.

In this book, I'm interested in showing you how to build cloud native applications with Spring and deploy them to Kubernetes as containers. Cloud native applications are distributed systems, just like microservices are. Some topics usually discussed in the context of microservices actually